

پروژه نهایی درس سیگنال ها و سیستم ها

طراحی و پیاده سازی موتورپردازش سیگنال یک ایستگاه کاری صوتی دیجیتال (DAW) در MATLAB

استاد: دکتر فاطمه رضایی

مبینا پورافشار

بهار 1405

فاز ۱. موتور سنتی سائزر و سنتز افزایشی

هدف این فاز

توی این فاز هدف این بود که با استفاده از **سری فوریه**، یک موج مثلثی را به صورت مصنوعی بسازیم و ببینیم چطور با اضافه کردن هارمونیک ها، شکل موج کم کم به فرم ایده آل خودش نزدیک می شود.

در واقع این فاز اولین بخش از پروژه است که در آن نقش **تجزیه و بازسازی سیگنال با مؤلفه های سینوسی** به صورت کاملاً عملی دیده می شود.

ایده ی اصلی این بود که به جای اینکه مستقیم یک موج مثلثی آماده تولید کنیم، آن را از جمع چند سینوس با فرکانس های مشخص بسازیم. این همان چیزی است که در **سنتز افزایشی** یا **Additive Synthesis** انجام می شود.

مبنای تئوری فاز

طبق صورت پروژه، موج مثلثی متقارن با دامنه $A=1$ و فرکانس پایه f_0 را می توان با سری فوریه ی زیر تقریب زد:

$$x_{tri}(t) = \frac{8}{\pi^2} \sum_{k \in \text{odd}} \frac{(-1)^{\frac{k-1}{2}}}{k^2} \sin(2\pi k f_0 t)$$

این رابطه چند نکته‌ی خیلی مهم را به ما می‌گوید:

- فقط هارمونیک‌های فرد در ساخت موج حضور دارند.
- دامنه‌ی هر هارمونیک با نسبت $\frac{1}{k^2}$ کم می‌شود.
- علامت جمله‌ها یکی‌درمیان عوض می‌شود که باعث شکل‌گیری فرم مثلثی می‌شود.
- هرچه تعداد هارمونیک‌های بیشتری نگه داریم، تقریب ما به موج مثلثی واقعی نزدیک‌تر می‌شود.

مشخصات پیاده‌سازی در MATLAB

در کد این فاز، پارامترهای اصلی مطابق صورت پروژه به شکل زیر انتخاب شدند:

نرخ نمونه‌برداری: **fs=44100 Hz** فرکانس پایه: **f0=440 Hz**

که همان نت لا است. مدت زمان سیگنال: **2 s**

بردار زمان هم به صورت گسسته از رابطه‌ی زیر ساخته شد:

$$t = 0, \frac{1}{f_s}, \frac{2}{f_s}, \dots, \left(2 - \frac{1}{f_s}\right)$$

این انتخاب باعث می‌شود سیگنال برای پردازش دیجیتال و ذخیره‌سازی صوتی کاملاً مناسب باشد.

سه حالت خواسته‌شده در صورت پروژه

حالت الف فقط هارمونیک اول: k=1

در این حالت عملاً فقط یک سینوس ساده با فرکانس 440 هرتز داریم.

حالت ب ترکیب ۵ هارمونیک فرد اول: k=1,3,5,7,9

در اینجا شکل موج از سینوس فاصله می‌گیرد و به موج مثلثی نزدیک‌تر می‌شود.

حالت ج ترکیب ۵۰ هارمونیک اول

در کد، این حالت با دنباله‌ی زیر ساخته شده است: **k=1,3,5,...,99**

چون از 1 تا 99 با گام 2 رفته‌ایم، دقیقاً 50 هارمونیک فرد خواهیم داشت.

توضیح روند کدنویسی

کد از سه بخش اصلی برای سنتز تشکیل شده است:

- ساخت سه سیگنال x_a ، x_b و x_c
- نرمال سازی دامنه
- ذخیره ی فایل های صوتی و رسم نمودارها

برای هر حالت، جمله های سری فوریه با یک حلقه به سیگنال اضافه شده اند.

ضریب ثابت جلوی سری هم به صورت زیر تعریف شده است: $8 / \pi^2$

و هر جمله ی هارمونیک به فرم زیر به سیگنال اضافه شده:

$$\frac{8}{\pi^2} \cdot \frac{(-1)^{\frac{k-1}{2}}}{k^2} \sin(2\pi k f_0 t)$$

این دقیقاً همان رابطه ی ریاضی صورت پروژه است که مستقیماً در MATLAB پیاده سازی شده.

نرمال سازی سیگنال ها

بعد از ساخت هر سه سیگنال، دامنه ی آن ها نرمال سازی شده تا در بازه ی $[-1,1]$ قرار بگیرند:

$$x[n] \leftarrow x[n] / \max|x[n]|$$

این کار دو دلیل مهم دارد:

- هنگام ذخیره ی فایل WAV از clipping جلوگیری می کند.
- مقایسه ی شنیداری بین سه حالت را منصفانه تر می کند.

یعنی تفاوتی که در شنیدن حس می شود بیشتر مربوط به رنگ صوتی و محتوای هارمونیک است، نه فقط بزرگ تر یا کوچک تر بودن دامنه.

خروجی های لازم این فاز

طبق صورت پروژه، در این فاز باید سه خروجی صوتی با فرمت WAV ذخیره می شد. این فایل ها در کد به درستی تولید شده اند:

- `x_a.wav`

- x_b.wav

- x_c.wav

همچنین دو نمودار اصلی هم تولید شده‌اند:

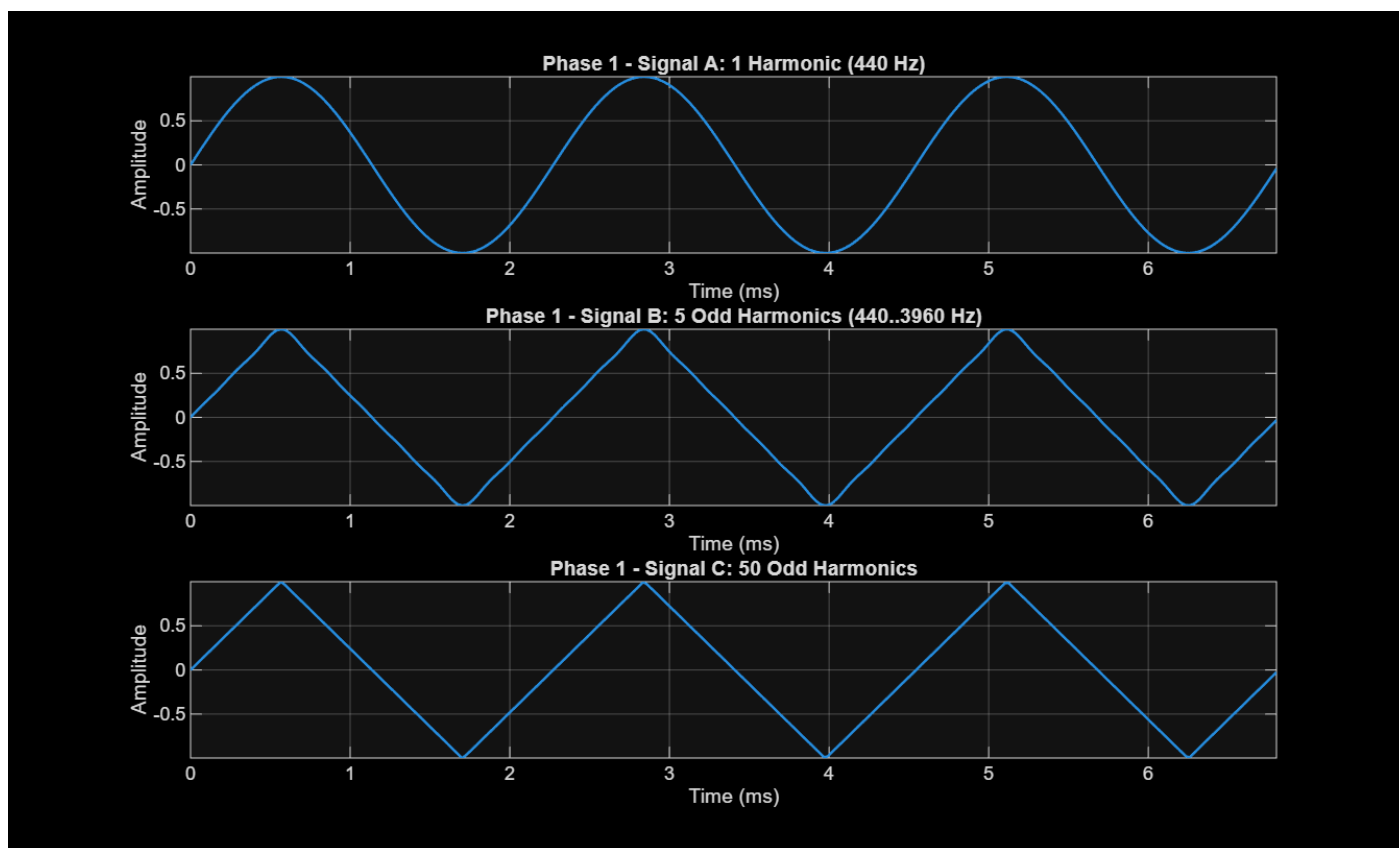
- Phase1_Time_Domain_Analysis.png

- Phase1_Frequency_Domain_Analysis.png

پس از نظر بخش خروجی، فاز ۱ کامل انجام شده است.

تحلیل حوزه زمان

در نمودار زمانی، سه سیگنال کنار هم رسم شده‌اند تا بتوانیم رفتارشان را مستقیم با هم مقایسه کنیم.



این نمودار فقط سه پریود اول را نشان می‌دهد تا شکل موج‌ها واضح دیده شوند.

حالت الف: فقط هارمونیک اول

در این حالت سیگنال فقط از یک مؤلفه‌ی سینوسی تشکیل شده، بنابراین شکل آن یک موج سینوسی کاملاً نرم و ساده است.

این سیگنال از نظر فرکانسی خالص‌ترین حالت را دارد، ولی هنوز هیچ شباهت جدی‌ای به موج مثلی ندارد.

برداشت

- شکل موج کاملاً نرم است
- هیچ گوشه یا شکست مشخصی ندارد
- فقط فرکانس پایه در آن وجود دارد

حالت ب: پنج هارمونیک فرد اول

در این حالت با اضافه شدن هارمونیک‌های فرد 3، 5، 7 و 9، شکل موج شروع می‌کند به نزدیک شدن به فرم مثلثی. دیگر موج مثل حالت قبل کاملاً گرد نیست و کم‌کم لبه‌ها و شیب‌های خطی‌تر در آن دیده می‌شود.

برداشت

- موج نسبت به سینوس خالص، غنی‌تر شده
- شباهت به مثلث بیشتر شده
- ولی هنوز گوشه‌ها کاملاً تیز و ایده‌آل نیستند

حالت ج: پنجاه هارمونیک فرد اول

در این حالت تعداد زیادی از هارمونیک‌های فرد وارد ترکیب شده‌اند و شکل سیگنال خیلی خوب به موج مثلثی نزدیک شده است. در نمودار زمانی هم واضح است که شیب‌ها تقریباً خطی شده‌اند و فرم کلی موج همان چیزی است که از یک موج مثلثی انتظار داریم.

برداشت

- تقریب موج مثلثی خیلی بهتر شده
- شکل نهایی بسیار نزدیک به فرم ایده‌آل است
- اثر افزایش تعداد هارمونیک‌ها کاملاً قابل مشاهده است

جمع‌بندی تحلیل زمانی

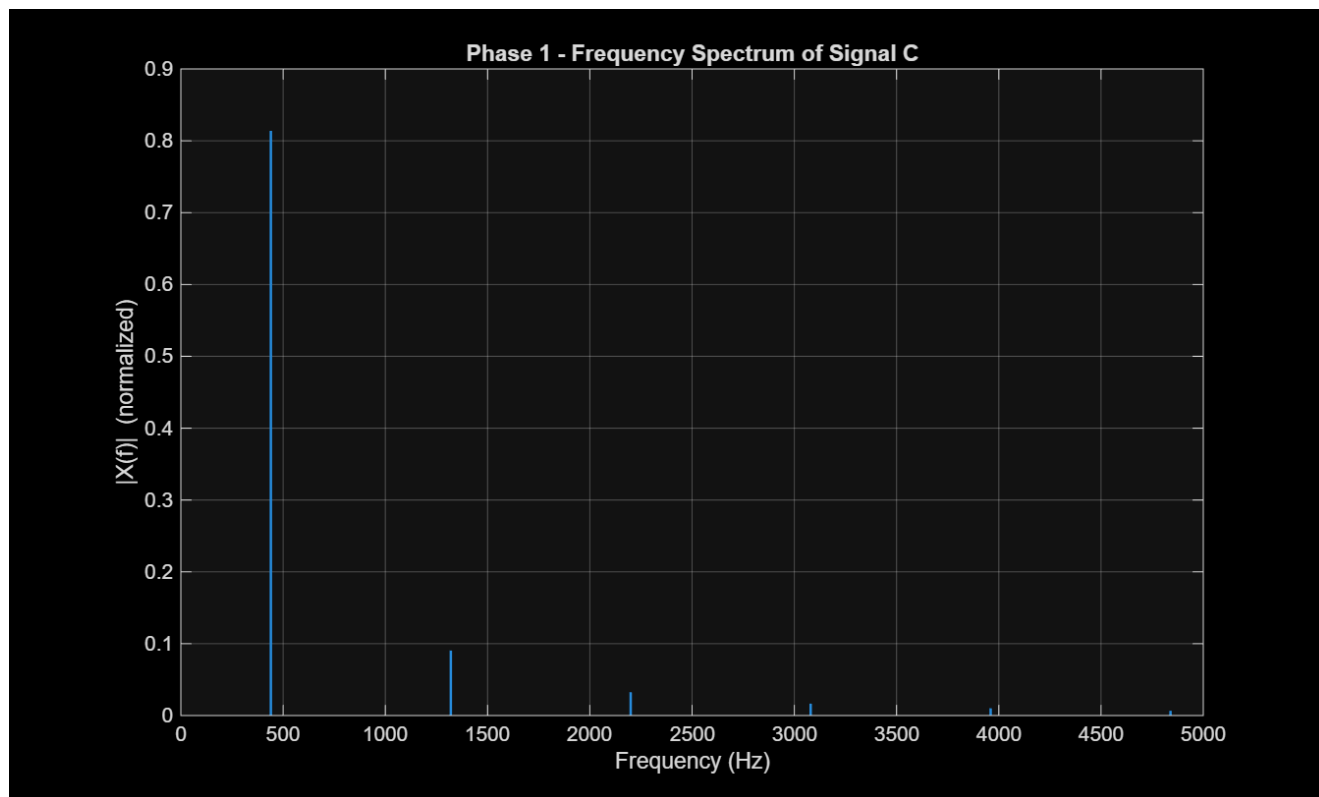
با مقایسه‌ی این سه نمودار می‌توان خیلی روشن دید که هرچه تعداد هارمونیک‌های فرد بیشتر می‌شود، شکل موج از یک سینوس ساده فاصله می‌گیرد و به موج مثلثی نزدیک‌تر می‌شود.

یعنی این فاز به صورت تصویری نشان می‌دهد که سری فوریه فقط یک رابطه‌ی تئوری نیست، بلکه واقعاً می‌تواند یک شکل موج پیچیده را از جمع مؤلفه‌های ساده بسازد.

تحلیل حوزه فرکانس

طبق صورت پروژه، باید روی سیگنال حالت ج، تبدیل فوریه‌ی سریع یا FFT اعمال می‌شد و طیف $|X(F)|$ رسم می‌شد.

در کد هم این کار روی x_c انجام شده و طیف یک‌طرفه رسم شده است.



آنچه در نمودار FFT دیده می‌شود:

در نمودار طیف فرکانسی سیگنال C، قله‌ها دقیقاً در مضارب فرد فرکانس پایه ظاهر شده‌اند.

چند قله‌ی اصلی که در شکل هم مشخص هستند عبارت‌اند از: 440, 1320, 2200, 3080, 3960, ...

که همگی از رابطه‌ی زیر به دست می‌آیند: $f_k = k f_0 \quad k=1,3,5,7,9,\dots$

از طرف دیگر، هرچه k بزرگ‌تر می‌شود، ارتفاع قله‌ها هم کمتر می‌شود.

این هم دقیقاً با انتظار تئوری سازگار است، چون دامنه‌ی هارمونیک‌ها باید متناسب با $\frac{1}{k^2}$ کاهش پیدا کند.

نتیجه‌ی مهم از تحلیل فرکانسی

این نمودار دو چیز را خیلی واضح تأیید می‌کند:

- فقط هارمونیک‌های فرد در سیگنال وجود دارند.
- دامنه‌ی هارمونیک‌ها با آهنگ تقریبی $\frac{1}{k^2}$ افت می‌کند.

بنابراین خروجی FFT با رابطه‌ی سری فوریه‌ی موج مثلثی کاملاً همخوانی دارد.

پاسخ سؤال ریاضی

در صورت پروژه از ما خواسته شده بود که **سری فوریه‌ی موج مثلثی متقارن** را از تعریف پایه به‌دست بیاوریم و نشان بدهیم:

- چرا فقط هارمونیک‌های فرد ظاهر می‌شوند
- و چرا دامنه‌ی آن‌ها با نسبت $\frac{1}{k^2}$ افت می‌کند

فرم کلی سری فوریه

برای یک سیگنال متناوب با دوره‌ی T ، سری فوریه به‌صورت کلی این‌طور نوشته می‌شود :

$$x(t) = a_0 + \sum_{k=1}^{\infty} a_k \cos(k\omega_0 t) + \sum_{k=1}^{\infty} b_k \sin(k\omega_0 t)$$

که در آن: $\omega_0 = 2\pi / T$

موج مثلثی متقارن یک سیگنال فرد است، بنابراین: $a_0=0, a_k=0$

و فقط جمله‌های سینوسی باقی می‌مانند. پس داریم:

$$x(t) = \sum_{k=1}^{\infty} b_k \sin(k\omega_0 t)$$

چرا فقط هارمونیک‌های فرد وجود دارند؟

موج مثلثی خاصیت تقارن نیم‌دوره‌ای دارد، یعنی: $x(t+T/2) = -x(t)$

این رابطه یکی از مهم‌ترین ویژگی‌های این موج است.

هر سیگنالی که این خاصیت را داشته باشد، هارمونیک‌های زوج آن حذف می‌شوند.

پس فقط جمله‌هایی باقی می‌مانند که شماره‌ی هارمونیکی آن‌ها فرد باشد.

یعنی در نهایت: $k=1,3,5,7,\dots$ در سری ظاهر می‌شوند.

چرا ضرایب با $\frac{1}{k^2}$ افت می‌کنند؟

دلیل اصلی این است که موج مثلی نسبت به موج مربعی هموارتر است.

موج مربعی پرش ناگهانی دارد، ولی موج مثلی فقط در شیبش شکست دارد و خودش پیوسته است.

در نتیجه ضرایب فوری‌ی آن سریع‌تر افت می‌کنند.

به‌طور شهودی:

- برای موج مربعی، ضرایب با $\frac{1}{K}$ افت می‌کنند
- برای موج مثلی، ضرایب با $\frac{1}{k^2}$ افت می‌کنند

این افت سریع‌تر به این معنی است که مؤلفه‌های فرکانسی بالاتر سهم کمتری در ساخت موج دارند، برای همین هم صدای موج مثلی نسبت به موج مربعی نرم‌تر شنیده می‌شود.

فرم نهایی ضرایب

با محاسبه‌ی ضرایب فوری، نتیجه‌ی نهایی این می‌شود:

$$b_k = \begin{cases} \frac{8}{\pi^2} \frac{(-1)^{\frac{k-1}{2}}}{k^2}, & k \text{ فرد} \\ 0, & k \text{ زوج} \end{cases}$$

پس رابطه‌ی کامل موج مثلی به شکل زیر خواهد بود:

$$x_{\text{tri}}(t) = \frac{8}{\pi^2} \sum_{k \in \text{odd}} \frac{(-1)^{\frac{k-1}{2}}}{k^2} \sin(2\pi k f_0 t)$$

این دقیقاً همان فرمولی است که در کد استفاده شده و اساس سنتز افزایشی این فاز را تشکیل می‌دهد.

تحلیل شنیداری سه حالت

یکی از بخش‌های مهم این فاز، مقایسه‌ی شنیداری تفاوت این سه سیگنال است، چون صورت پروژه صریحاً به تفاوت **تیمبر** یا **رنگ صدا** اشاره کرده است.

حالت الف: در این حالت فقط هارمونیک اول وجود دارد، بنابراین صدا خیلی خالص و ساده شنیده می‌شود.

این صدا از نظر گوش تقریباً شبیه یک **tone** سینوسی تمیز است و بافت هارمونیک زیادی ندارد.

حالت ب: با اضافه شدن ۵ هارمونیک فرد اول، صدا پرت‌تر و غنی‌تر می‌شود.

هنوز نرم است، ولی دیگر آن خلوص کامل حالت اول را ندارد و شخصیت صوتی مشخص‌تری پیدا می‌کند.

حالت ج: در این حالت چون ۵۰ هارمونیک فرد در ساخت سیگنال نقش دارند، صدا بیشترین شباهت را به موج مثلثی پیدا می‌کند.

از نظر شنیداری، این حالت نسبت به دو حالت قبلی کامل‌تر و طبیعی‌تر شنیده می‌شود و رنگ صوتی‌اش تثبیت شده‌تر است.

جمع‌بندی شنیداری

هرچه تعداد هارمونیک‌ها بیشتر می‌شود:

- صدا از حالت خیلی ساده فاصله می‌گیرد
- رنگ صوتی غنی‌تر می‌شود
- و خروجی بیشتر به شخصیت صوتی موج مثلثی نزدیک می‌شود

فاز ۲. نمونه‌برداری، تخریب سیگنال و فیلتر آنتی‌الیاسینگ

هدف این فاز

توی این فاز هدف این بود که یک **صدای واقعی** را اول با نرخ نمونه‌برداری بالا ضبط کنیم، بعد آن را به یک نرخ پایین‌تر تبدیل کنیم و ببینیم اگر این کار را **بدون فیلتر** انجام بدهیم چه اتفاقی برای سیگنال می‌افتد.

در واقع این فاز برای این طراحی شده بود که اثر **aliasing** یا همان **آلیاسینگ** را به صورت عملی ببینیم و بعد نشان بدهیم که اگر قبل از **downsampling** از یک **فیلتر آنتی‌الیاسینگ** مناسب استفاده کنیم، کیفیت سیگنال بهتر می‌شود و تخریب طیفی کمتر اتفاق می‌افتد.

پس خلاصه‌ی ایده این بود:

- یک سیگنال گفتاری واقعی ضبط کنیم
- آن را از نرخ بالای 44100 هرتز به نرخ پایین‌تر 8000 هرتز کاهش بدهیم
- یک بار بدون فیلتر
- یک بار با فیلتر پایین‌گذر
- و بعد خروجی‌ها را هم در حوزه‌ی زمان و هم در حوزه‌ی فرکانس مقایسه کنیم

مشخصات اصلی پیاده‌سازی

پارامترهای اصلی این فاز در کد به این شکل انتخاب شدند:

- نرخ نمونه‌برداری اولیه: $fs1=44100 \text{ Hz}$

- نرخ نمونه‌برداری هدف: $fs2=8000 \text{ Hz}$

- مدت زمان ضبط: $T=10 \text{ s}$

- فرکانس قطع فیلتر آنتی‌الیاسینگ: $fc=3800 \text{ Hz}$

- مرتبه‌ی فیلتر Butterworth: $N=8$

این انتخاب‌ها کاملاً با منطق پروژه هماهنگ هستند، چون نرخ 44100 هرتز برای ضبط صدا کیفیت مناسبی می‌دهد و نرخ 8000 هرتز هم نرخ پایین‌تری است که برای شبیه‌سازی کانال محدود یا سیستم کم‌نرخ استفاده می‌شود.

ورودی این فاز

در این فاز به جای تولید سیگنال مصنوعی، یک صدای واقعی ضبط شد.

طبق کد، سیگنال مرجع با تابع `record_audio(fs1, duration)` گرفته شده است.

متن ضبط‌شده هم همان جمله‌ی معرفی بوده :

«سلام من مبینا پورافشار به شماره دانشجویی 40318893 دانشجوی رشته مهندسی کامپیوتر دانشگاه خواجه نصیر هستم»

این کار باعث می‌شود سیگنال ورودی، به یک سیگنال واقعی گفتاری نزدیک باشد و تحلیل aliasing فقط روی یک سیگنال تئوری نباشد، بلکه روی داده‌ی واقعی هم دیده شود.

ایده‌ی اصلی downsampling در این فاز

در این پروژه، سیگنال باید از نرخ 44100 هرتز به 8000 هرتز کاهش پیدا می‌کند.

نکته‌ی مهم این است که نسبت این دو نرخ عدد صحیح نیست: $8000 = 5.5125 / 44100$

یعنی نمی‌شود خیلی ساده هر 5 یا هر 6 نمونه را برداشت و گفت دقیقاً به 8000 هرتز رسیده‌ایم.

برای همین در کد، با استفاده از فاصله‌ی نمونه‌برداری تقریبی و اندیس‌های زمانی، سیگنال به شکل مناسب نمونه‌برداری مجدد شده است. در عمل دو حالت بررسی شد:

حالت اول: downsampling بدون فیلتر

در این حالت سیگنال مستقیم و بدون هیچ آماده‌سازی قبلی به نرخ پایین‌تر منتقل شد.

این روش ساده است، اما چون مؤلفه‌های فرکانسی بالای سیگنال قبل از کاهش نرخ حذف نمی‌شوند، باعث aliasing می‌شود.

حالت دوم: downsampling همراه با فیلتر آنتی‌الیاسینگ

در این حالت اول سیگنال از یک فیلتر پایین‌گذر Butterworth عبور داده شد و بعد downsample شد.

این روش درست‌تر است، چون مؤلفه‌های فرکانسی خطرناک قبل از ورود به نرخ پایین حذف یا تضعیف می‌شوند.

چرا آلیاسینگ رخ می‌دهد؟

وقتی نرخ نمونه‌برداری را کم می‌کنیم، بازه‌ی فرکانسی قابل‌نمایش هم کوچک‌تر می‌شود.

فرکانس نایکوئیست برای جدید 8000 هرتز برابر است با: $2 = 4000 \text{ Hz} / f_N = f_s$

یعنی بعد از کاهش نرخ، فقط فرکانس‌های بین 0 تا 4000 هرتز را می‌توانیم بدون ابهام نمایش بدهیم.

اگر قبل از downsampling، مؤلفه‌هایی بالاتر از این مقدار در سیگنال وجود داشته باشند، بعد از نمونه‌برداری به پایین تا می‌خورند و به‌صورت فرکانس‌های اشتباه دیده می‌شوند. به این پدیده می‌گویند aliasing.

رابطه‌ی ریاضی آلیاسینگ

برای پاسخ ریاضی، می‌توان فرکانس alias را به‌صورت زیر نوشت: $f_{alias} = |f - k f_s|$

که در آن:

- f فرکانس اصلی است
- f_s نرخ نمونه برداری است
- k یک عدد صحیح است که باعث می شود فرکانس نهایی داخل بازه ی مجاز قرار بگیرد

این رابطه نشان می دهد که اگر یک مؤلفه ی فرکانسی خارج از بازه ی نایکوئیست وجود داشته باشد، بعد از نمونه برداری به یک فرکانس دیگر نگاشت می شود و در نتیجه طیف سیگنال خراب می شود.

چرا فرکانس قطع را 3800 هرتز گرفتیم؟

ما نرخ هدف را 8000 هرتز گرفتیم، پس فرکانس نایکوئیست می شود 4000 هرتز.

از نظر تئوری، فیلتر آنتی الیاسینگ باید قبل از رسیدن به این مرز شروع به تضعیف کند، نه دقیقاً روی مرز.

برای همین فرکانس قطع را 3800 هرتز انتخاب کردیم تا یک **حاشیه ی امن** قبل از 4000 هرتز داشته باشیم.

چرا این حاشیه لازم است؟

چون فیلتر واقعی مثل فیلتر ایده آل نیست.

فیلتر واقعی ناحیه ی گذار دارد و ناگهان از عبور کامل به قطع کامل نمی رسد.

اگر فرکانس قطع را دقیقاً روی 4000 هرتز بگذاریم، ممکن است هنوز بخشی از مؤلفه های خطرناک عبور کنند و aliasing رخ بدهد.

برای همین 3800 هرتز انتخاب عاقلانه تری است.

طراحی فیلتر آنتی الیاسینگ

برای این فاز از یک فیلتر پایین گذر Butterworth با مرتبه ی 8 استفاده شد: $N=8$

فیلتر Butterworth به این دلیل مناسب است که در باند عبور، پاسخ نسبتاً صاف و بدون ریبِل دارد.

یعنی برای سیگنال صوتی انتخاب خوبی است، چون قرار نیست در فرکانس های عبوری نوسان یا اعوجاج شدید ایجاد کند.

تابع انتقال این فیلتر به صورت کلی چنین است: $H(z) = B(z) / A(z)$

و در کد با دستور زیر ساخته شده است:

```
[b, a] = butter(order, fc/(fs1/2), 'low');
```

که در آن:

- `order = 8`
- `fc = 3800`
- `fs1 = 44100`

پس فرکانس نرمال شده‌ی قطع برابر است با:

$$\frac{f_c}{f_{s1}/2} = \frac{3800}{22050}$$

چرا از `filtfilt` استفاده شد؟

در کد، فیلتر کردن با دستور زیر انجام شده است: `x_filtered = filtfilt(b, a, x_original);`

این انتخاب مهم است، چون `filtfilt` فیلتر را **دو بار** و در دو جهت اعمال می‌کند و در نتیجه:

- فاز سیگنال را جابه‌جا نمی‌کند
- تأخیر فازی ایجاد نمی‌شود
- شکل موج تمیزتر باقی می‌ماند

برای تحلیل این پروژه این موضوع خیلی خوب است، چون می‌خواهیم اثر آلیاسینگ را ببینیم، نه اینکه تغییر فاز فیلتر باعث سردرگمی شود.

خروجی‌های این فاز

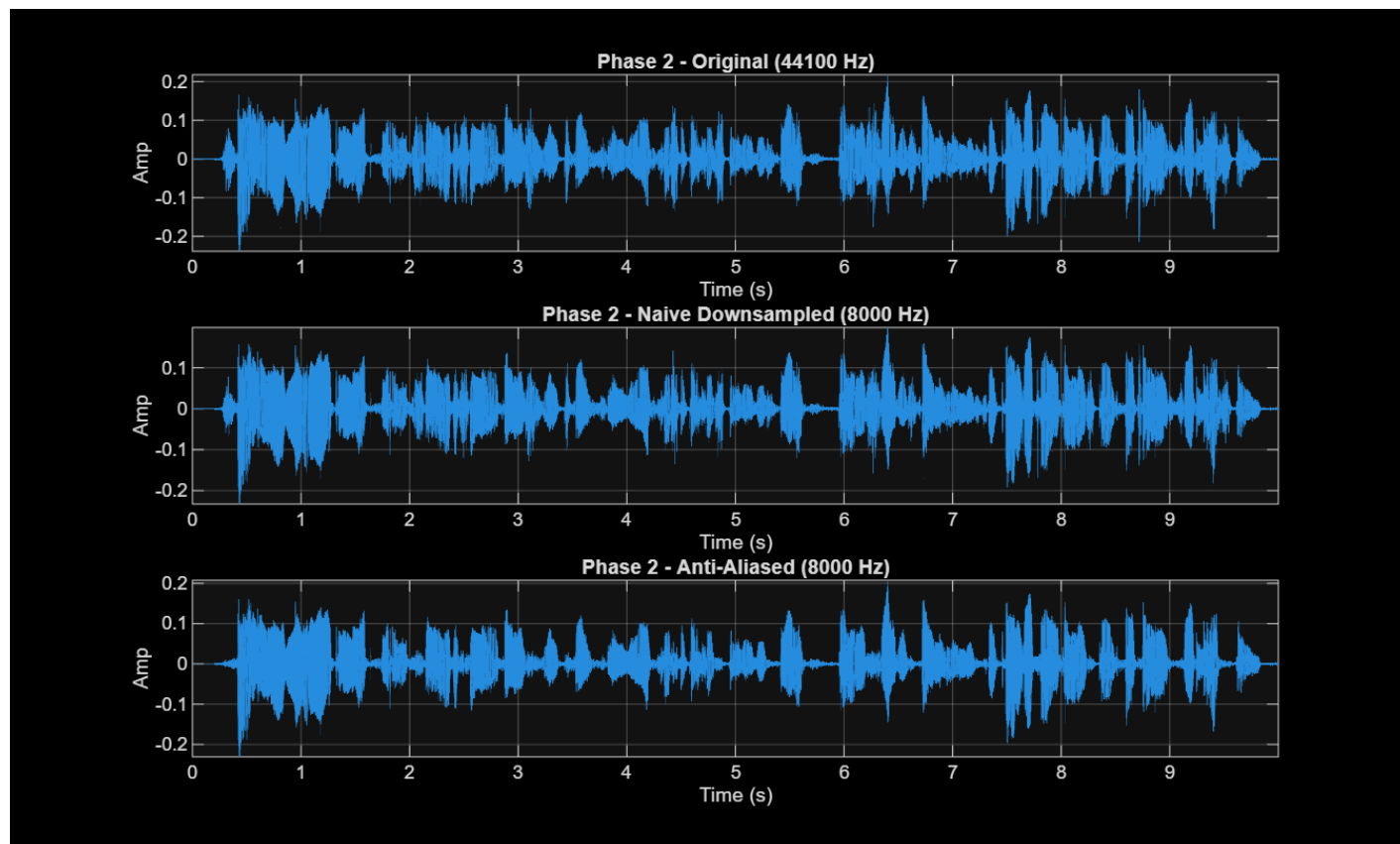
طبق خواسته‌ی پروژه، سه فایل صوتی تولید شدند:

- `x_original.wav`
- `x_naive.wav`
- `x_antialiased.wav`

همچنین دو نمودار اصلی ذخیره شدند:

- `Phase2_Time_Domain_Analysis.png`
- `Phase2_Frequency_Domain_Analysis.png`

پس از نظر خروجی، این فاز کامل است.



1. سیگنال اصلی با نرخ 44100 هرتز
2. سیگنال downsample شده‌ی بدون فیلتر
3. سیگنال downsample شده‌ی همراه با فیلتر آنتی‌الیاسینگ

نتیجه‌ی دیداری در حوزه‌ی زمان

در نگاه اول، شکل کلی هر سه نمودار شبیه هم است، چون محتوای گفتار هنوز حفظ شده است. اما تفاوت اصلی در جزئیات و کیفیت سیگنال است.

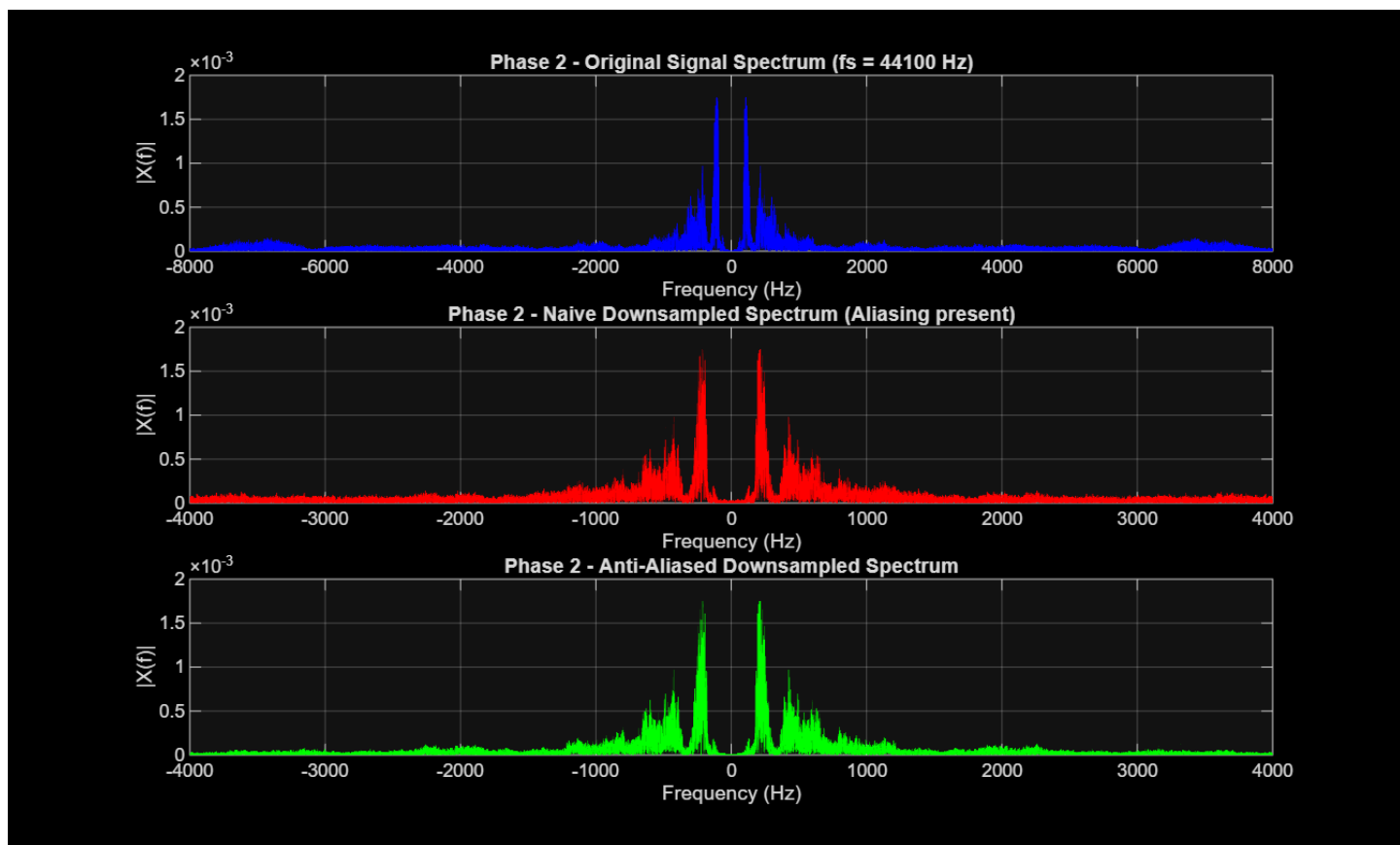
در سیگنال بدون فیلتر، بعضی از مؤلفه‌های پرفرکانس به صورت پنهان وارد باند پایین‌تر شده‌اند و این موضوع در زمان به شکل اعوجاج‌های ریز و ناپیوستگی‌های نامحسوس دیده می‌شود. در سیگنال فیلترشده، شکل کلی طبیعی‌تر و تمیزتر است.

برداشت از حوزه‌ی زمان

- سیگنال اصلی کامل‌ترین نسخه است
- سیگنال naive کمی آسیب‌دیده
- سیگنال anti-aliased تمیزتر و قابل‌اعتمادتر است

تحلیل حوزه‌ی فرکانس

در نمودار فرکانسی، سه طیف کنار هم رسم شده‌اند و با استفاده از `fftshift`، طیف حول صفر متمرکز شده است.



طیف سیگنال اصلی

در سیگنال اصلی، پهنای باند فرکانسی وسیع‌تر است و مؤلفه‌ها تا فرکانس‌های بالاتر دیده می‌شوند.

این کاملاً طبیعی است، چون صدای گفتار واقعی معمولاً شامل انرژی در بازه‌های مختلف فرکانسی است.

طیف سیگنال naive

در خروجی بدون فیلتر، بخشی از مؤلفه‌های بالای 4000 هرتز به داخل باند اصلی تا خورده‌اند.

به همین دلیل، طیف شلوغ‌تر به نظر می‌رسد و ساختار واقعی بعضی از مؤلفه‌ها از بین می‌رود.

این دقیقاً همان اثر `aliasing` است:

یعنی فرکانس‌هایی که نباید در سیگنال جدید دیده شوند، به فرکانس‌های دیگر تبدیل شده‌اند و باعث خراب شدن طیف شده‌اند.

طیف سیگنال anti-aliased

در خروجی فیلترشده، مؤلفه‌های فرکانسی بالا قبل از downsampling تضعیف شده‌اند.

به همین دلیل، طیف مرتب‌تر است و تداخل فرکانسی کمتری دیده می‌شود.

این یعنی فیلتر آنتی‌الیاسینگ کار خودش را درست انجام داده است.

چرا در خروجی بدون فیلتر کیفیت شنیداری پایین‌تر است؟

چون aliasing فقط یک مسئله‌ی تئوری نیست؛ مستقیماً روی صدای شنیده‌شده اثر می‌گذارد.

وقتی مؤلفه‌های بالا به اشتباه وارد باند پایین می‌شوند، سیگنال حالت طبیعی خود را از دست می‌دهد و صدا کمی خشن‌تر، ناصاف‌تر و مصنوعی‌تر شنیده می‌شود.

در مقابل، در سیگنال فیلترشده:

- نویز ناشی از آلیاسینگ کمتر است
- گفتار طبیعی‌تر شنیده می‌شود
- و کیفیت کلی بهتر است

نقش تابع centeredSpectrum

در کد برای تحلیل طیفی از تابع کمکی centeredSpectrum استفاده شده است.

این تابع سه کار انجام می‌دهد:

1. FFT سیگنال را می‌گیرد
2. با fftshift طیف را حول صفر مرکزی می‌کند
3. اندازه‌ی طیف را نرمال‌سازی می‌کند

فرم کلی آن به این شکل است:

$$\text{mag} = \text{abs}(\text{fftshift}(\text{fft}(x))) / N;$$

این نرمال‌سازی کمک می‌کند که مقایسه‌ی طیف‌ها راحت‌تر شود و فقط شکل کلی و محل انرژی‌ها دیده شود، نه اینکه اختلاف مقیاس باعث گمراهی شود.

سؤال‌های مهم

سؤال 1: چرا downsampling بدون فیلتر مشکل ساز است؟

چون قبل از کاهش نرخ، مؤلفه‌های فرکانسی بالای سیگنال حذف نمی‌شوند. وقتی نرخ نمونه‌برداری کم می‌شود، این مؤلفه‌های بالا به داخل باند مجاز تا می‌خورند و باعث aliasing می‌شوند. در نتیجه طیف و صدای خروجی خراب می‌شود.

سؤال 2: چرا باید قبل از downsampling فیلتر پایین‌گذر بگذاریم؟

چون هدف این است که فقط مؤلفه‌هایی را نگه داریم که بعد از کاهش نرخ هنوز قابل نمایش هستند. فیلتر پایین‌گذر جلوی عبور فرکانس‌های خطرناک را می‌گیرد و اجازه نمی‌دهد آن‌ها در نرخ جدید به فرکانس‌های اشتباه تبدیل شوند.

سؤال 3: چرا فیلتر Butterworth انتخاب شد؟

چون پاسخ آن در باند عبور صاف است و ریبِل ندارد. برای صدا، این ویژگی مهم است، چون می‌خواهیم سیگنال طبیعی‌تر بماند و خود فیلتر کمترین اعوجاج اضافی را ایجاد کند.

سؤال 4: چرا cutoff را 3800 هرتز گرفتید و نه 4000 هرتز؟

چون فرکانس نایکوئیست برای نرخ جدید 4000 هرتز است، اما فیلتر واقعی ایده‌آل نیست و ناحیه‌ی گذار دارد. بنابراین باید کمی زودتر شروع به تضعیف کند تا مؤلفه‌های خطرناک قبل از مرز نایکوئیست حذف شوند.

سؤال 5: چرا از filtfilt استفاده شد؟

چون filtfilt فیلتر را بدون ایجاد تأخیر فازی اعمال می‌کند و شکل سیگنال را طبیعی‌تر نگه می‌دارد. این باعث می‌شود برای مقایسه‌ی aliasing، نتیجه تمیزتر و دقیق‌تر باشد.

سؤال 6: چرا aliasing در حوزه‌ی فرکانس دیده می‌شود ولی در زمان ممکن است خیلی واضح نباشد؟

چون بعضی از اثرات aliasing در ظاهر موج زمان خیلی شدید نیستند، اما در ساختار فرکانسی کاملاً مشخص‌اند. برای همین تحلیل فرکانسی در این فاز خیلی مهم است و عملاً نشان می‌دهد چه اتفاقی برای سیگنال افتاده است.

تحلیل شنیداری خروجی‌ها

فایل x_original.wav

این فایل صدای مرجع است و طبیعی‌ترین نسخه را نشان می‌دهد، چون با نرخ بالا ضبط شده است.

فایل x_naive.wav

این فایل به‌خاطر نبود فیلتر قبل از downsampling، کمی خشن‌تر و همراه با اعوجاج شنیده می‌شود.

علت اصلی این است که aliasing وارد سیگنال شده و بخشی از اطلاعات فرکانسی اشتباه به گوش می‌رسد.

فایل x_antialiased.wav

این نسخه تمیزتر و طبیعی‌تر شنیده می‌شود، چون فیلتر پایین‌گذر، مؤلفه‌های اضافی را قبل از کاهش نرخ حذف کرده است.

فاز ۱.۳ کوپویا با کانولوشن و فیلتر بازگشتی (FIR و IIR)

هدف این فاز

توی این فاز هدف این بود که خروجی فازهای قبل را وارد یک محیط صوتی جذاب‌تر کنیم و بعد روی آن **اکو** بسازیم.

یعنی دیگر فقط بحث تحلیل سیگنال نبود، بلکه می‌خواستیم ببینیم اگر یک صدای گفتاری تمیز را با یک موسیقی غنی ترکیب کنیم و بعد روی این ترکیب اکو اعمال کنیم، خروجی چه شکلی می‌شود.

از طرف دیگر، این فاز برای این طراحی شده بود که تفاوت دو نگاه مهم در درس سیگنال و سیستم‌ها را به‌صورت عملی ببینیم:

- **FIR** که با کانولوشن مستقیم پیاده‌سازی می‌شود
- **IIR** که با رابطه‌ی بازگشتی و فیلتر دیجیتال ساخته می‌شود

همچنین یک بخش خیلی مهم این فاز این بود که تفاوت **سیستم پایدار** و **ناپایدار** را در عمل ببینیم.

ایده‌ی اصلی پروژه در این فاز

در این فاز سه کار اصلی انجام شد:

مرحله اول

صدای تمیز و ضدآلیاس شده‌ی فاز ۲ یعنی فایل `x_antialiased.wav` به عنوان **صدای اصلی گفتار** استفاده شد.

مرحله دوم

صدای موسیقی غنی فاز ۱ یعنی فایل `x_c.wav` به عنوان **بخش موسیقی** وارد شد.

مرحله سوم

موسیقی قبل از ترکیب، با یک **پوش نمایی نزولی** تضعیف شد تا کم‌کم از شدت آن کم شود و صدای گفتار واضح‌تر بماند. بعد از این کار، یک ترکیب استودیویی ساخته شد و روی آن اکو به دو روش اعمال شد:

- روش **FIR** با استفاده از **کانولوشن**
- روش **IIR** با استفاده از **رابطه‌ی بازگشتی**

در پایان هم یک نسخه‌ی **پایدار** و یک نسخه‌ی **ناپایدار** از IIR ساخته شد تا تفاوتشان دیده شود.

فایل‌ها و خروجی‌های این فاز

فایل‌های صوتی

- `x_studio.wav`
- `y_fir_stable.wav`
- `y_iir_stable.wav`
- `y_iir_unstable.wav`

نمودارها

- `Phase3_FIR_Impulse_Response.png`
- `Phase3_IIR_Stability_Comparison.png`

پس از نظر خروجی، فاز ۳ کامل است و هم فایل صوتی دارد، هم تحلیل تصویری.

نرخ نمونه‌برداری کاری

در کل این فاز با نرخ نمونه‌برداری: **$fs=8000$ Hz** انجام شده است.

این انتخاب کاملاً منطقی است، چون صدای گفتاری فاز ۲ هم روی همین نرخ قرار داشت و برای پردازش اکو، یک نرخ یکسان لازم بود تا همه چیز هماهنگ بماند.

ساخت سیگنال ترکیبی (Voice + Music)

صدای گفتار

صدای گفتار از فایل x_antialiased.wav خوانده شد.

اگر نرخ آن با 8000 هرتز یکی نباشد، با resample به نرخ موردنظر تبدیل می‌شود.

سپس فقط یک کانال از آن گرفته می‌شود تا سیگنال **مونو** باشد.

موسیقی

صدای موسیقی از فایل x_c.wav گرفته شد که در فاز ۱ با سنتز هارمونیک‌ها ساخته شده بود.

این سیگنال هم به نرخ 8000 هرتز تبدیل شد.

تضعیف نمایی موسیقی

برای اینکه موسیقی در ابتدا حضور داشته باشد ولی کم‌کم عقب برود و گفتار واضح‌تر شنیده شود، یک پوش نمایی تعریف شد:

$$\text{env}(t) = e^{-t/\tau}$$

که در آن: $\tau = 1.5 \text{ s}$ این یعنی موسیقی با گذر زمان آرام‌آرام کم می‌شود.

چرا این کار انجام شد؟

چون اگر موسیقی از اول تا آخر با شدت ثابت بماند، ممکن است صدای گفتار را بپوشاند.

ولی با این روش، ترکیب حالت طبیعی‌تری پیدا می‌کند؛ اول فضای موسیقایی داریم و بعد تمرکز بیشتر روی صدا می‌رود.

نرمال سازی و میکس

قبل از ترکیب نهایی، هر دو سیگنال نرمال شدند تا یکی به‌خاطر دامنه‌ی بیشتر، دیگری را تحت‌الشعاع قرار ندهد.

سپس ترکیب نهایی به شکل زیر ساخته شد:

$$\text{xstudio} = g\text{Voice} \cdot \text{voice} + g\text{Music} \cdot \text{musicdamped}$$

که ضرایب آن در کد چنین‌اند: $g\text{Voice} = 0.90$, $g\text{Music} = 0.35$

دلیل این انتخاب

چون در این پروژه باید صدای گفتار واضح و قابل فهم باقی بماند.

موسیقی نقش پس‌زمینه دارد، نه صدای اصلی.

به همین خاطر بهره‌ی گفتار بیشتر از موسیقی در نظر گرفته شده است.

جلوگیری از clipping

بعد از میکس، سیگنال نهایی دوباره نرمال شد تا از بریدگی دامنه جلوگیری شود.

مفهوم اکو در این فاز

اکو یعنی یک نسخه‌ی تأخیردار و تضعیف‌شده از سیگنال اصلی را به خود سیگنال اضافه کنیم.

اگر این کار را چند بار تکرار کنیم، چند بازتاب پشت سرهم از صدا می‌شنویم.

در این پروژه، اکو به صورت چندتپی ساخته شده است، یعنی چند بار تکرار با فاصله‌ی یکسان:

- هر بار با تأخیر ثابت
- هر بار با ضریب کاهش دامنه

پارامترهای اکو

تأخیر اکو : $R = \text{round}(fs \cdot 0.4)$

چون $fs = 8000$ است، داریم: $R = 8000 \times 0.4 = 3200$

یعنی تأخیر اکو برابر 0.4 ثانیه است.

تعداد تکرار : $\text{numTaps} = 5$

یعنی سیگنال اصلی و چهار بازتاب بعد از آن در نظر گرفته شده است.

ضریب تضعیف پایدار : $\alpha_{st} = 0.6$

ضریب ناپایدار : $\alpha_{un} = 1.05$

بخش FIR

در مدل FIR، اکو به صورت جمع چند ضربه‌ی تأخیری ساخته می‌شود.

تابع ضربه‌ی این سیستم به شکل زیر است:

$$h[n] = \sum_{i=0}^4 \alpha^i \delta[n - iR]$$

یعنی:

- در $n=0$ یک ضربه با دامنه‌ی 1
- در $n=R$ یک ضربه با دامنه‌ی α
- در $n=2R$ یک ضربه با دامنه‌ی α^2
- و همین‌طور تا $n=4R$

معنی این فرمول: یعنی صدا بعد از هر 0.4 ثانیه یک بار تکرار می‌شود و هر بار ضعیف‌تر می‌شود.

چرا این سیستم FIR است؟

چون پاسخ ضربه‌ی آن **طول محدود** دارد.

یعنی بعد از آخرین ضربه، دیگر هیچ انرژی‌ای در پاسخ ضربه وجود ندارد.

به همین دلیل به آن می‌گویند: **FIR=FiniteImpulseResponse**

پیاده‌سازی FIR در کد

در کد، بردار پاسخ ضربه با طول:

$$(\text{numTaps}-1)R+1$$

ساخته شده و روی اندیس‌های مناسب، مقادیر α^i قرار داده شده‌اند.

بعد از آن، خروجی با کانولوشن مستقیم محاسبه شده است:

$$y_{\text{fir}} = x_{\text{studio}} * h$$

چرا از کانولوشن استفاده شد؟

چون سیستم FIR دقیقاً با کانولوشن تعریف می‌شود.

این روش از نظر مفهومی خیلی شفاف است و به خوبی نشان می‌دهد که اکو یعنی جمع نسخه‌های تأخیردار سیگنال.

نتیجه‌ی FIR

خروجی FIR یک صدای واضح با چند بازتاب منظم است.

چون $\alpha = 0.6$ است، هر بازتاب ضعیف‌تر از قبلی می‌شود و سیستم پایدار و کنترل‌شده باقی می‌ماند.

بخش IIR

در مدل IIR، اکو فقط از روی ورودی فعلی ساخته نمی‌شود، بلکه خروجی‌های قبلی هم دوباره وارد محاسبه می‌شوند.

رابطه‌ی بازگشتی این سیستم به شکل زیر است: $y[n] = x[n] + \alpha y[n-R]$

این یعنی خروجی فعلی برابر است با ورودی فعلی به علاوه‌ی یک نسخه‌ی تأخیردار از خروجی قبلی.

تابع انتقال IIR

تابع انتقال این سیستم برابر است با:

$$H(z) = 1 / (1 - (\alpha z^{-R}))$$

این فرمول خیلی مهم است، چون نشان می‌دهد که سیستم یک فیدبک دارد و همین فیدبک می‌تواند باعث پایداری یا ناپایداری شود.

چرا این سیستم IIR است؟

چون پاسخ ضربه‌ی آن بی‌نهایت طول است.

در واقع چون خروجی خودش دوباره وارد حلقه می‌شود، پاسخ سیستم به صورت نظری تمام نمی‌شود.

به همین دلیل به آن می‌گویند: **IIR=InfinitelmpulseResponse**

تحلیل پایداری IIR

شرط کلی پایداری

برای اینکه سیستم بازگشتی پایدار باشد، باید ضریب فیدبک از نظر قدرمطلق کوچکتر از 1 باشد: $|\alpha| < 1$

اگر این شرط برقرار باشد، بازتابها در هر دور ضعیفتر می‌شوند و سیستم bounded می‌ماند.

حالت پایدار در این پروژه

در حالت پایدار داریم: $\alpha_{st} = 0.6$

چون: $|0.6| < 1$

پس سیستم پایدار است.

نتیجه

- خروجی رشد نمی‌کند
- دامنه کنترل شده می‌ماند
- اکوها کم‌کم محو می‌شوند

حالت ناپایدار در این پروژه

در حالت ناپایدار داریم: $\alpha_{un} = 1.05$

چون: $|1.05| > 1$

پس سیستم ناپایدار است.

نتیجه

- هر بار بازخورد از دفعه‌ی قبل بزرگ‌تر می‌شود
- دامنه‌ی خروجی رشد می‌کند
- سیگنال از کنترل خارج می‌شود

چرا در خروجی ناپایدار clipping انجام شد؟

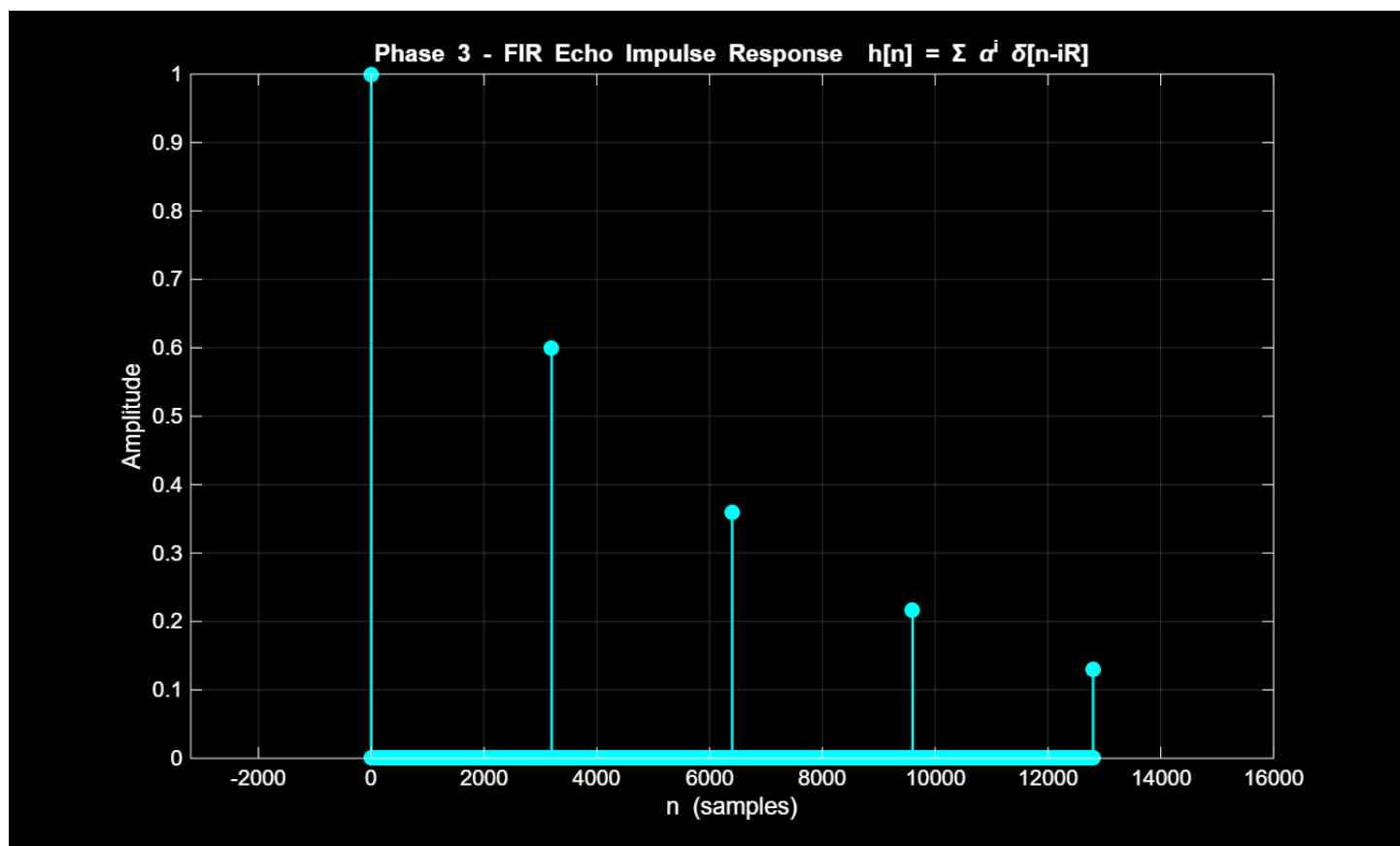
در کد، خروجی ناپایدار نرمال نشد تا رشد واقعی آن دیده شود.

اما برای شبیه‌سازی شرایط واقعی، مقدار سیگنال در بازه‌ی $[-1.1, 1.1]$ محدود شد تا مثل یک سیستم سخت‌افزاری که اشباع می‌شود، رفتار آن قابل شنیدن باشد.

این کار خیلی خوب است، چون نشان می‌دهد ناپایداری در دنیای واقعی به اعوجاج شدید و اشباع می‌رسد.

تحلیل شکل پاسخ ضربه‌ی FIR

در فایل Phase3_FIR_Impulse_Response.png، پاسخ ضربه به صورت stem plot رسم شده است.



در این نمودار، چند ضربه در فواصل منظم دیده می‌شود:

- در 0
- در R
- در 2R
- در 3R
- در 4R

و ارتفاع هر ضربه برابر است با:

$$1, \alpha, \alpha^2, \alpha^3, \alpha^4$$

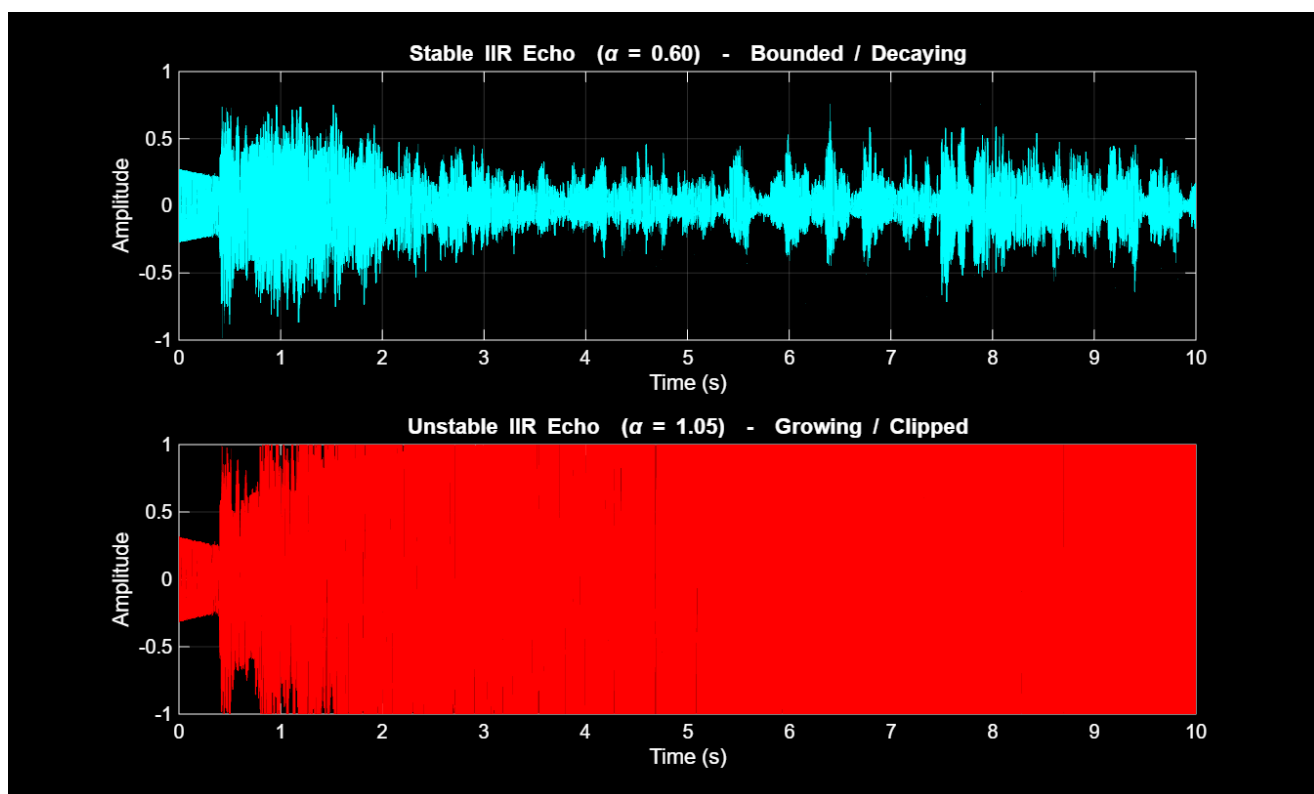
نتیجه‌ی تحلیلی

این نمودار دقیقاً نشان می‌دهد که سیستم چگونه اکو را می‌سازد:

یعنی سیگنال اصلی را با چند نسخه‌ی کپی‌شده و تأخیردار از خودش ترکیب می‌کند.

تحلیل شکل مقایسه‌ی پایداری IIR

در فایل Phase3_IIR_Stability_Comparison.png دو نمودار دیده می‌شود:



نمودار اول: حالت پایدار

در این حالت، خروجی در محدوده‌ی قابل کنترل باقی می‌ماند و با گذر زمان بازتاب‌ها ضعیف‌تر می‌شوند.

نمودار دوم: حالت ناپایدار

در این حالت، خروجی رشد می‌کند و در نهایت به اشباع می‌رسد.

این شکل یکی از بهترین شواهد عملی برای فهم مفهوم پایداری است.

یعنی فقط با دیدن نمودار می‌توان فهمید که وقتی α از 1 عبور کند، سیستم دیگر قابل اعتماد نیست.

چرا موسیقی را قبل از میکس تضعیف کردیم؟

اگر موسیقی با قدرت زیاد وارد می‌شد، گفتار را می‌پوشاند و فهم آن سخت می‌شد.

ولی چون هدف پروژه این بود که هم **اثر موسیقی** حس شود و هم **وضوح گفتار** حفظ شود، از پوشش نمایی نزولی استفاده شد.

پس دلیلش این بود که:

- فضای صوتی جذاب بماند
- ولی صدای اصلی گم نشود

این تصمیم از نظر شنیداری کاملاً منطقی است و نشان می‌دهد طراحی سیگنال فقط محاسبه نیست، بلکه درک کاربردی هم می‌خواهد.

چرا delay برابر 0.4 ثانیه انتخاب شد؟

چون اگر تأخیر خیلی کوچک باشد، اکو از بازتاب طبیعی خیلی فاصله می‌گیرد و بیشتر شبیه تکرار سریع یا flanging می‌شود.

اگر هم خیلی بزرگ باشد، ارتباط بین نسخه‌های اکو زیاد از بین می‌رود.

مقدار 0.4 ثانیه ثانیه برای اکو، یک مقدار مناسب و قابل شنیدن است؛ هم واضح است، هم طبیعی.

پاسخ سؤالات مهم

سؤال 1: تفاوت FIR و IIR چیست؟

در FIR، خروجی فقط به ورودی‌های فعلی و گذشته وابسته است و پاسخ ضربه طول محدود دارد.

اما در IIR، خروجی به خروجی‌های قبلی هم وابسته است و پاسخ ضربه از نظر نظری طول بی‌نهایت دارد.

سؤال 2: چرا FIR در این پروژه با کانولوشن پیاده‌سازی شد؟

چون FIR ذاتاً با کانولوشن تعریف می‌شود.

اگر در این مدل یعنی جمع چند نسخه‌ی تأخیردار و ضعیف‌شده‌ی سیگنال، و این دقیقاً همان چیزی است که کانولوشن انجام می‌دهد.

سؤال 3: تابع انتقال IIR این پروژه چیست؟

تابع انتقال برابر است با: $H(z) = 1 / (1 - \alpha z^{-R})$

که در آن R تأخیر و α ضریب بازخورد است.

سؤال 4: شرط پایداری IIR چیست؟

باید داشته باشیم: $|\alpha| < 1$

اگر این شرط برقرار باشد، بازخوردها ضعیف می‌شوند و سیستم پایدار می‌ماند.

سؤال 5: چرا حالت $\alpha = 1.05$ ناپایدار است؟

چون بازخورد از 1 بزرگ‌تر است و هر بار نسخه‌ی تأخیردار خروجی، از قبل بزرگ‌تر می‌شود.

در نتیجه خروجی رشد می‌کند و سیستم از کنترل خارج می‌شود.

سؤال 6: چرا در حالت ناپایدار سیگنال را clip کردید؟

برای اینکه رفتار ناپایدار در یک خروجی واقعی‌تر و قابل شنیدن نشان داده شود.

در دنیای واقعی هم سیستم‌های سخت‌افزاری به اشباع می‌رسند، بنابراین clipping کمک می‌کند اثر ناپایداری ملموس‌تر شود.

سؤال 7: چرا از صدای فاز ۲ و موسیقی فاز ۱ استفاده شد؟

چون هدف این فاز این بود که خروجی‌های مراحل قبل دوباره وارد یک پردازش جدید شوند تا پروژه یک روند پیوسته داشته باشد.

همچنین ترکیب گفتار تمیز با موسیقی غنی، برای نمایش اکو و فیدبک خیلی مناسب است.

فاز ۴. مهندسی فرکانس و تحلیل صفحه Z — اکولایزر ۳ بانده

هدف این فاز

توی این فاز هدف این بود که خروجی فاز ۳، یعنی سیگنال اکوخورده‌ی پایدار از نوع FIR، وارد یک اکولایزر ۳ بانده بشود تا بتوانیم بخش‌های مختلف طیف صدا را جداگانه کنترل کنیم.

ایده‌ی اصلی این بود که صدا را به سه ناحیه‌ی فرکانسی تقسیم کنیم:

- باند پایین یا **Bass**
- باند میانی یا **Mid**
- باند بالا یا **Treble**

بعد از جدا کردن این سه باند، برای هر کدام یک ضریب تقویت جدا در نظر گرفته شد تا بتوانیم دو حالت صوتی مختلف بسازیم:

- حالت **Bass Boost**
- حالت **Muffled**

علاوه بر تولید خروجی صوتی، این فاز از ما می‌خواست که رفتار فیلترها را هم از چند زاویه بررسی کنیم؛ یعنی هم پاسخ فرکانسی آن‌ها را ببینیم، هم پاسخ ضربه را تحلیل کنیم، هم مکان صفر و قطب‌ها را در صفحه Z رسم کنیم و در نهایت پایداری فیلترها را بررسی کنیم.

ورودی فاز ۴

طبق صورت پروژه، ورودی این فاز باید خروجی پایدار FIR از فاز ۳ باشد.

برای همین در کد، فایل زیر به عنوان ورودی استفاده شد: **y_fir_stable.wav**

این فایل ابتدا با **audioread** خوانده شد و اگر نرخ نمونه‌برداری آن با نرخ کاری این فاز یکی نبود، با استفاده از **resample** به نرخ مناسب تبدیل شد.

نرخ نمونه‌برداری کاری در این فاز برابر با: **fs=8000 Hz**

در نتیجه فرکانس نایکوئیست برابر است با: **fNyquist=fs/2=4000 Hz**

این موضوع خیلی مهم است، چون تمام طراحی فیلترها باید نسبت به همین بازه‌ی فرکانسی انجام شود.

ایده‌ی کلی طراحی اکولایزر

ساختاری که برای اکولایزر استفاده شد، ساختار موازی بود.

یعنی سیگنال ورودی به جای اینکه از چند فیلتر به صورت سری عبور کند، به سه شاخه تقسیم شد و هر شاخه از یک فیلتر جدا عبور کرد:

- شاخه‌ی بم: فیلتر پایین‌گذر
- شاخه‌ی میانی: فیلتر میان‌گذر
- شاخه‌ی زیر: فیلتر بالاگذر

بعد از فیلتر شدن، این سه خروجی با ضرایب دلخواه وزن‌دهی شدند و با هم جمع شدند تا خروجی نهایی ساخته شود.

رابطه‌ی کلی اکولایزر به صورت زیر است: $y[n] = G_{bass} y_{bass}[n] + G_{mid} y_{mid}[n] + G_{treble} y_{treble}[n]$ که در آن:

- $y_{bass}[n]$ خروجی شاخه‌ی پایین‌گذر است
- $y_{mid}[n]$ خروجی شاخه‌ی میان‌گذر است
- $y_{treble}[n]$ خروجی شاخه‌ی بالاگذر است
- و ضرایب G_{bass} ، G_{mid} و G_{treble} شدت هر باند را کنترل می‌کنند

طراحی فیلترها

برای هر سه باند از فیلترهای دیجیتال IIR از نوع **Butterworth** استفاده شد.

دلیل انتخاب Butterworth این است که این فیلتر در باند عبور پاسخ نسبتاً صاف و بدون نوسان دارد و برای کاربردهای صوتی انتخاب مناسبی است، چون تغییر ناگهانی و مصنوعی در باند عبور ایجاد نمی‌کند.

سه فیلتر طراحی شده به این صورت بودند:

الف) فیلتر Bass : یک فیلتر پایین‌گذر Butterworth با فرکانس قطع: $f_c = 300 \text{ Hz}$

ب) فیلتر Mid : یک فیلتر میان‌گذر Butterworth با بازه‌ی عبور: $300 \text{ Hz} \leq f \leq 2000 \text{ Hz}$

ج) فیلتر Treble : یک فیلتر بالاگذر Butterworth با فرکانس قطع: $f_c = 2000 \text{ Hz}$

*نکته‌ی مهم درباره مرتبه‌ی فیلتر میان‌گذر

صورت پروژه خواسته بود که هر سه فیلتر مرتبه ۴ باشند. برای فیلتر پایین‌گذر و بالاگذر مستقیماً از **butter(4,...)** استفاده شد، اما برای فیلتر میان‌گذر از دستور زیر استفاده شد:

[b_mid, a_mid] = butter(2, band_mid/fnyq, 'bandpass');

ممکن است در نگاه اول اینطور به نظر برسد که این فیلتر مرتبه ۲ است، ولی در MATLAB وقتی از bandpass استفاده می‌کنیم، مرتبه‌ی کلی فیلتر برابر با $2N$ می‌شود. پس اگر: $N=2$ باشد، مرتبه‌ی نهایی فیلتر میان‌گذر برابر می‌شود با: $2N=4$

پس این طراحی کاملاً با خواسته‌ی صورت پروژه منطبق است.

پیاده‌سازی اکولایزر موازی

بعد از طراحی ضرایب، سیگنال ورودی از هر سه فیلتر عبور داده شد:

- خروجی فیلتر پایین‌گذر: **ybass[n]**
- خروجی فیلتر میان‌گذر: **ymid[n]**
- خروجی فیلتر بالاگذر: **ytreble[n]**

بعد برای دو حالت مختلف، ضرایب اکولایزر مطابق صورت پروژه تنظیم شدند.

حالت اول: Bass Boost

در این حالت ضرایب به شکل زیر در نظر گرفته شدند: **Gbass=2.5, Gmid=1.0, Gtreble=0.5**

در این تنظیم، باند پایین تقویت می‌شود، باند میانی تقریباً حفظ می‌شود و باند بالا کمی تضعیف می‌شود.

در نتیجه انتظار داریم صدا گرم‌تر، پرتر و بم‌تر شنیده شود.

حالت دوم: Muffled

در این حالت ضرایب به شکل زیر هستند: **Gbass=2.0, Gmid=0.1, Gtreble=0.1**

در اینجا باند پایین هنوز نسبتاً قوی نگه داشته می‌شود، اما باند میانی و بالا به شدت تضعیف می‌شوند.

پس خروجی باید حالتی خفه، پشت دیواری و گرفته داشته باشد.

نرمال سازی خروجی ها

بعد از ساخت خروجی نهایی، هر دو سیگنال برای جلوگیری از اشباع دامنه نرمال سازی شدند.

این کار با این هدف انجام شد که هنگام ذخیره سازی فایل WAV، مقدار نمونه ها از بازه ی مجاز خارج نشود و clipping اتفاق نیفتد. فرم کلی این نرمال سازی به این صورت بود:

$$y_{\text{norm}}[n] = \frac{y[n]}{\max(|y[n]| + \varepsilon)} \times 0.98$$

این نرمال سازی صرفاً برای جلوگیری از اعوجاج دامنه در فایل خروجی است و ماهیت فیلترها یا ساختار اکولایزر را تغییر نمی دهد.

فایل های خروجی این فاز

طبق خواسته ی پروژه، دو فایل صوتی نهایی تولید شدند:

1. y_bass_boost.wav
2. y_muffled.wav

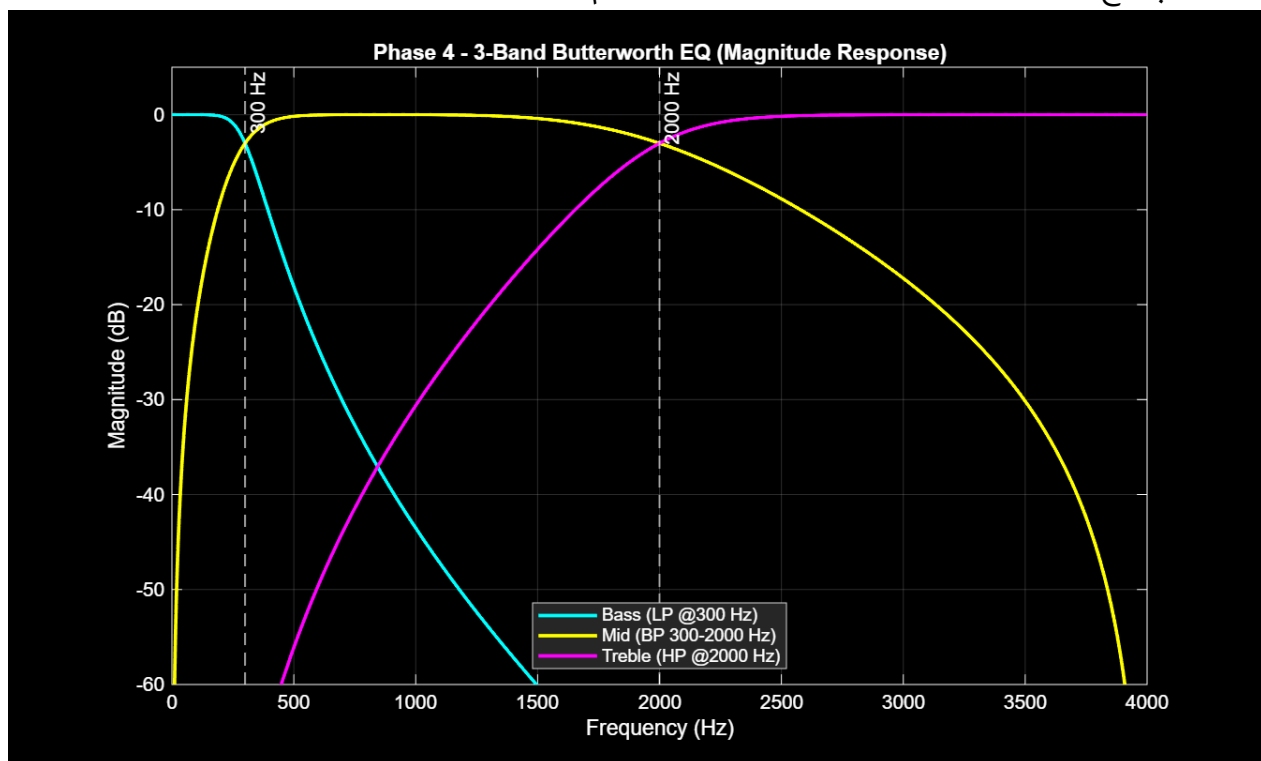
همچنین چهار شکل تحلیلی زیر ذخیره شدند:

1. Phase4_Filter_Frequency_Responses.png
2. Phase4_Output_Spectrum_Comparison.png
3. Phase4_Impulse_Responses.png
4. Phase4_PoleZero_Maps.png

بنابراین از نظر بخش خروجی، این فاز کامل انجام شده است.

تحلیل نمودار پاسخ فرکانسی فیلترها

در شکل پاسخ فرکانسی، هر سه فیلتر روی یک نمودار رسم شدند تا تفکیک باندها بهتر دیده شود.



تحلیل شکل

- منحنی آبی روشن مربوط به فیلتر **Bass** است. این فیلتر تا حدود 300 هرتز باند پایین را عبور می‌دهد و بعد از آن شروع به افت می‌کند.
- منحنی زرد مربوط به فیلتر **Mid** است. این فیلتر در بازه‌ی حدود 300 تا 2000 هرتز بیشترین عبور را دارد و بیرون از این بازه تضعیف می‌شود.
- منحنی صورتی مربوط به فیلتر **Treble** است. این فیلتر مؤلفه‌های فرکانسی بالاتر از حدود 2000 هرتز را عبور می‌دهد.

دو خط عمودی در 300 هرتز و 2000 هرتز هم روی نمودار رسم شدند تا مرز سه باند کاملاً مشخص باشد.

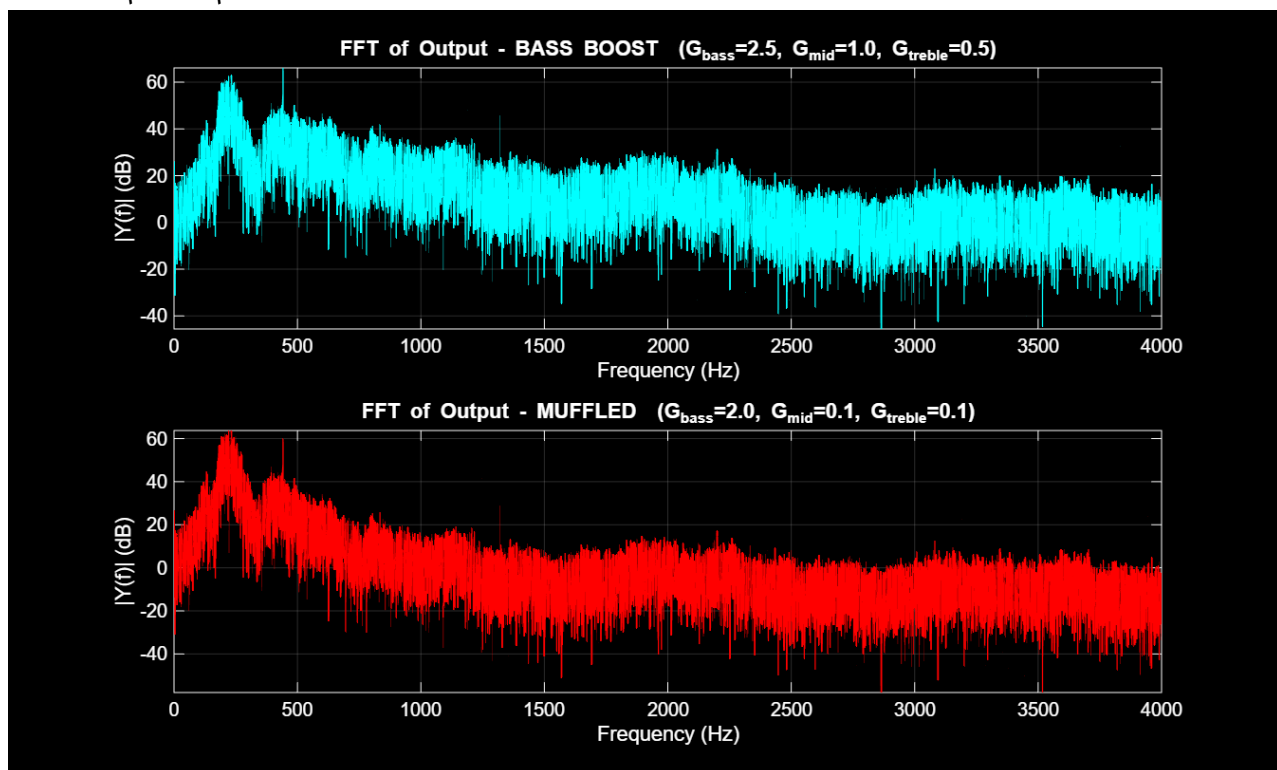
برداشت نهایی از این نمودار

این شکل نشان می‌دهد که اکولایزر واقعاً سه ناحیه‌ی فرکانسی مورد نیاز را از هم جدا کرده و طراحی فیلترها مطابق خواسته‌ی پروژه انجام شده است.

از طرفی چون فیلترها از نوع Butterworth هستند، گذارها نسبتاً نرم و بدون نوسان شدید دیده می‌شوند که برای پردازش صوت مطلوب است.

تحلیل طیف خروجی نهایی با FFT

در این بخش، طیف خروجی دو preset یعنی **Bass Boost** و **Muffled** با استفاده از FFT رسم و با هم مقایسه شد.



حالت Bass Boost

در این نمودار دیده می‌شود که در ناحیه‌ی فرکانس‌های پایین، دامنه‌ی مؤلفه‌ها نسبتاً بالاتر است.

این دقیقاً با هدف این preset هماهنگ است، چون ضریب باند Bass برابر 2.5 در نظر گرفته شده و از همه بزرگ‌تر است.

در نتیجه صدای خروجی در این حالت باید:

- بم‌تر
- پرحجم‌تر
- و تا حدی گرم‌تر

شنیده شود.

حالت Muffled

در این حالت به‌خاطر اینکه: $G_{mid}=0.1$, $G_{treble}=0.1$

بخش‌های میانی و بالای طیف به شدت افت کرده‌اند.

به همین دلیل نمودار FFT این حالت نسبت به Bass Boost در ناحیه‌های میانی و بالا ضعیف‌تر دیده می‌شود.

این یعنی خروجی:

- وضوح کمتری دارد
- جزئیات فرکانس‌های بالا کمتر شنیده می‌شود
- و صدا حالت خفه یا پشت مانع پیدا می‌کند

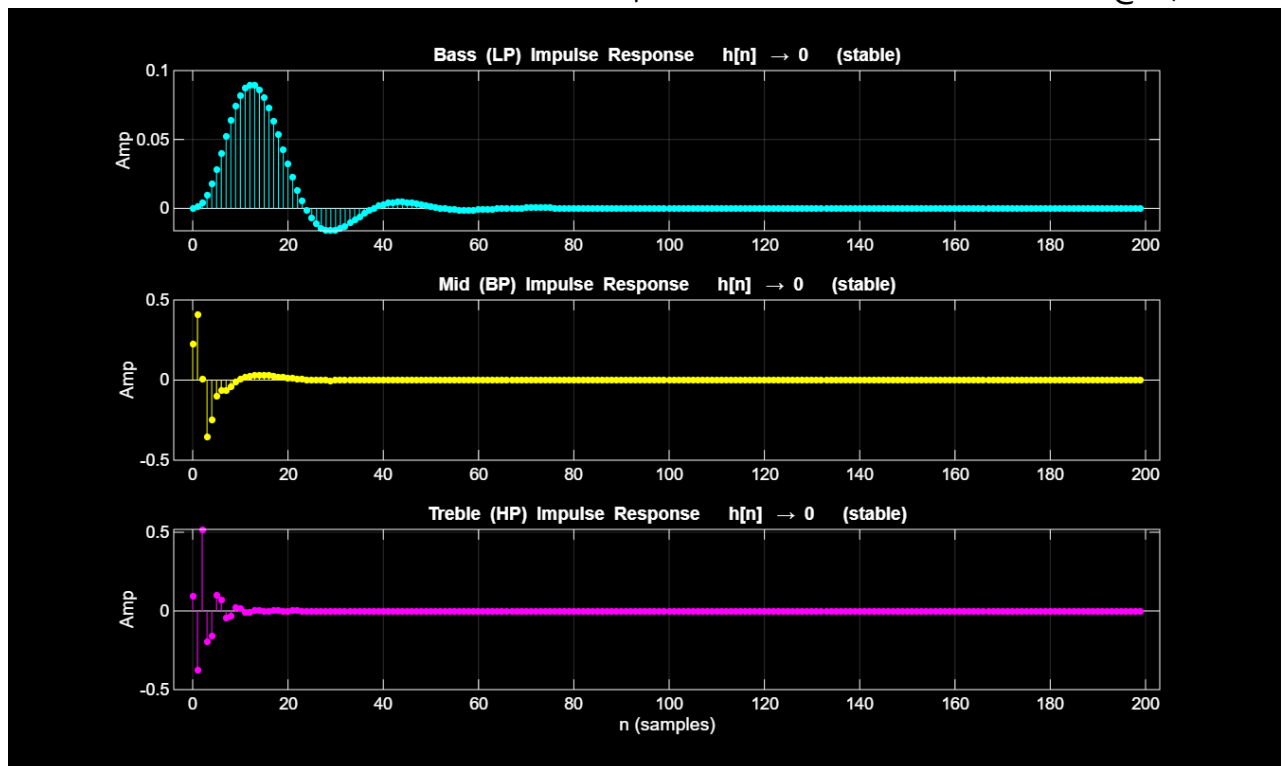
جمع‌بندی تحلیلی این بخش

FFT خروجی‌ها دقیقاً رفتار مورد انتظار دو preset را تأیید می‌کند.

یعنی تفاوت شنیداری این دو حالت فقط ادعای ذهنی نیست، بلکه در حوزه‌ی فرکانس هم کاملاً قابل مشاهده است.

تحلیل پاسخ ضربه $h[n]$

در شکل پاسخ ضربه، برای هر سه فیلتر نمودار impz رسم شد.



- پاسخ ضربه‌ی هر سه فیلتر در ابتدا مقدار قابل توجهی دارد
- بعد از چند نمونه، دامنه‌ی نوسانات کم‌کم کاهش پیدا می‌کند
- و در نهایت به صفر میل می‌کند

این رفتار برای یک فیلتر IIR پایدار طبیعی است.

یعنی سیستم ممکن است چند نمونه اثر حافظه داشته باشد، اما پاسخ ضربه‌اش واگرا نمی‌شود و به مرور میرا می‌شود.

نتیجه‌گیری

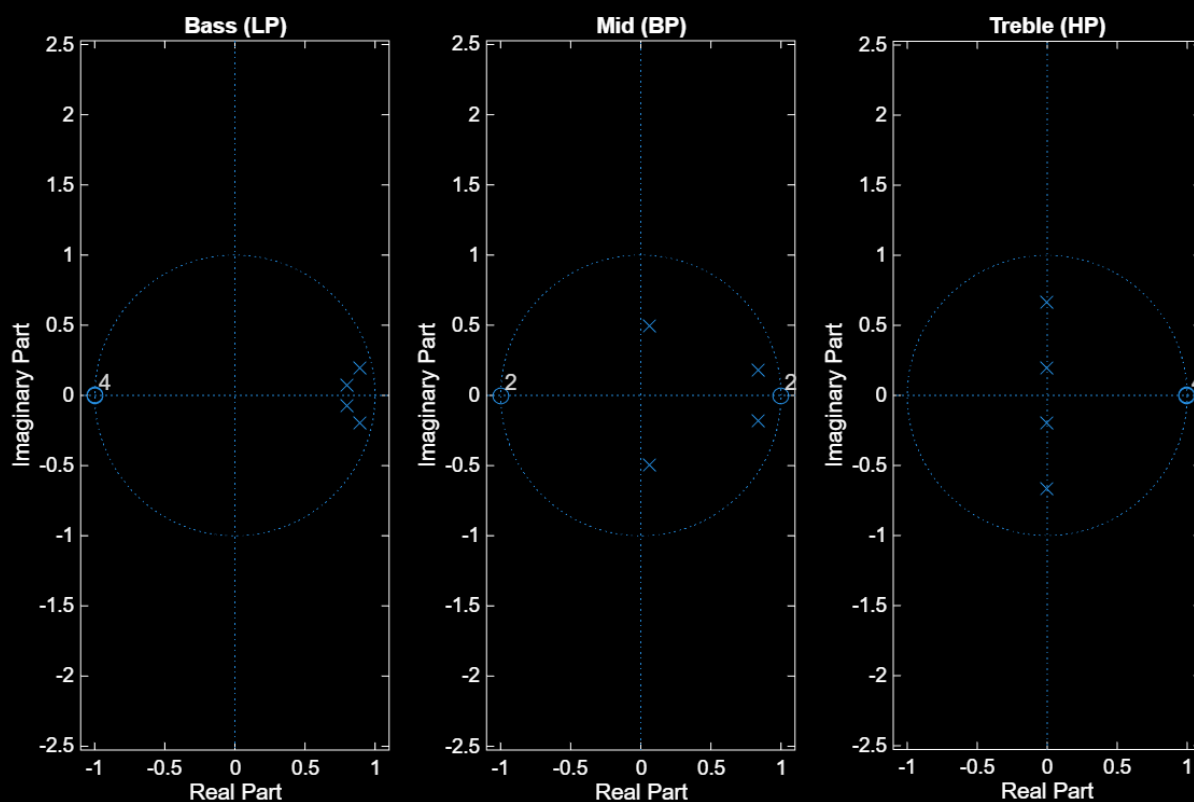
چون برای هر سه فیلتر داریم: $h[n] \rightarrow 0$ as $n \rightarrow \infty$

پس از دید زمانی هم رفتار سیستم‌ها پایدار است.

این نتیجه بعداً با تحلیل قطب‌ها در صفحه Z هم تأیید می‌شود.

تحلیل صفحه Z و پایداری

در شکل zplane، صفرها و قطب‌های هر سه فیلتر رسم شدند.



قاعده‌ی پایداری BIBO برای سیستم‌های LTI گسسته این است که تمامی قطب‌ها باید داخل دایره واحد قرار داشته باشند.

یعنی اگر برای تمامی قطب‌ها داشته باشیم: $|p_i| < 1$ سیستم پایدار است.

نتیجه‌ی عددی از اجرای کد

طبق خروجی Command Window:

- برای فیلتر Bass: $\max|p|=0.914320$
- برای فیلتر Mid: $\max|p|=0.856618$
- برای فیلتر Treble: $\max|p|=0.668179$

همه‌ی این مقادیر از 1 کوچک‌ترند، پس نتیجه می‌گیریم: $|p_i| < 1$ for all poles

بنابراین هر سه فیلتر BIBO Stable هستند.

برداشت از روی شکل

در شکل صفحه Z هم محل قطب‌ها کاملاً داخل دایره واحد دیده می‌شود.

پس هم از نظر عددی و هم از نظر بصری، پایداری فیلترها تأیید می‌شود.

تابع تبدیل ریاضی فیلتر Bass

طبق صورت پروژه، لازم بود تابع تبدیل $H(z)$ برای فیلتر پایین‌گذر از روی ضرایب a و b نوشته شود.

ضرایب به‌دست‌آمده از اجرای کد به صورت زیر هستند:

ضرایب صورت: $b = [0.0001441202, 0.0005764809, 0.0008647213, 0.0005764809, 0.0001441202]$

ضرایب مخرج: $a = [1.0000000000, -3.3849727283, 4.3370618174, -2.4891382938, 0.5393551283]$

پس تابع تبدیل فیلتر Bass برابر است با: $H_{\text{bass}}(z)$

$$= \frac{0.0001441202 + 0.0005764809z^{-1} + 0.0008647213z^{-2} + 0.0005764809z^{-3} + 0.0001441202z^{-4}}{1 - 3.3849727283z^{-1} + 4.3370618174z^{-2} - 2.4891382938z^{-3} + 0.5393551283z^{-4}}$$

این تابع تبدیل نشان می‌دهد که فیلتر Bass یک سیستم IIR مرتبه ۴ است، چون مخرج آن علاوه بر صورت، جمله‌های وابسته به z^{-1} تا z^{-4} دارد.

چرا ساختار موازی برای این پروژه مناسب بوده است؟

در این پروژه از ساختار **موازی** استفاده شد، نه ساختار سری. چرا؟ چون در اکولایزر، معمولاً هدف این است که باندهای مختلف فرکانسی را جداگانه کنترل کنیم.

در ساختار موازی، هر شاخه مسئول یک بخش از طیف است و بعد از فیلتر کردن، می‌توانیم خیلی راحت برای همان باند یک ضریب جدا اعمال کنیم.

اگر فیلترها را به صورت سری می‌گذاشتیم:

- خروجی هر مرحله ورودی مرحله‌ی بعد می‌شد
- کنترل مستقل روی باندها سخت‌تر می‌شد
- و تفسیر نقش هر فیلتر در خروجی نهایی به آن سادگی نبود

پس برای یک اکولایزر ۳ بانده، ساختار موازی هم از نظر مفهومی روشن‌تر است و هم از نظر پیاده‌سازی مناسب‌تر.

تحلیل شنیداری دو خروجی

Bass Boost (الف)

در این حالت، چون باند پایین تقویت شده و باند بالا کمی کاهش پیدا کرده، خروجی صدایی بم‌تر و قوی‌تر دارد. در شنیدن این فایل، معمولاً حس می‌شود که:

- بخش‌های کم‌فرکانس برجسته‌تر شده‌اند
- صدا کمی ضخیم‌تر و پرت‌تر شده
- و در بعضی بخش‌ها وضوح خیلی بالا کمتر از حالت طبیعی است، چون تریبل کاهش یافته

این preset بیشتر مناسب حالتی است که بخواهیم ضرب، بیس یا حس سنگینی صدا بیشتر شنیده شود.

Muffled (ب)

در این حالت، چون باندهای میانی و بالا خیلی ضعیف شده‌اند، خروجی به شکل واضح خفه‌تر شنیده می‌شود. در این فایل:

- وضوح گفتار کمتر می‌شود
- حروف تیز و صداهای روشن کمتر شنیده می‌شوند
- و حس کلی شبیه صدایی است که از پشت دیوار یا از پشت یک مانع شنیده شود

پس از نظر شنیداری هم رفتار خروجی‌ها دقیقاً مطابق هدف طراحی بوده است. (: